1a)

```
procedure:  InsertionSortRec (L[0:n-1])
input:      L[0: n-1]  (a list of size n)
output:     L[0: n-1]  (a sorted list)

    if n ≤ 1  then return L[0:n-1]
    InsertionSortRec ( L[0: n-1])
    lastVal ← L[n-1]
    currIndex ← n-2
    while currIndex ≥ 0  and  L[currIndex] > lastVal
        L[currIndex +1] ← L[currIndex]
        currIndex ← currIndex - 1
    end while
    L[currIndex + 1] ← lastVal

end InsertionSortRec
```

1b)

```
procedure:  InsertionSortRecL (head)
input:      head  (head of linked list)
output:     head  (head node of the sorted linked list)

    if head == NULL  or  head.next == NULL
        return head
    newH ← InsertionSortRecL (head.next)
    head.next ← NULL
    if head.value < newH.value
        head.next ← newH
        return head
    else
        curr ← newH;
        while curr.next != NULL   curr ← curr.next
        head.next ← curr.next ← head; return newH
```

2)

$$\lim_{n\to\infty} \frac{10}{\ln(n)} = 10 \lim_{n\to\infty} \frac{1}{\ln(n)} = 0$$

$$\lim_{n\to\infty} \frac{\ln(n)}{n} = \lim_{n\to\infty} \frac{\frac{1}{n}}{1} = 0$$

$$\lim_{n\to\infty} \frac{\cancel{n}}{\cancel{n}\ln(n)} = \lim_{n\to\infty} \frac{1}{\ln(n)} = 0$$

$$\lim_{n\to\infty} \frac{\cancel{n}\ln(n)}{n^2} = \lim_{n\to\infty} \frac{\ln(n)}{n} = \lim_{n\to\infty} \frac{\frac{1}{n}}{1} = 0$$

$$\lim_{n\to\infty} \frac{\cancel{n^2}}{\cancel{n^3}} = \lim_{n\to\infty} \frac{1}{n} = 0$$

$$\lim_{n\to\infty} \frac{n^3}{2^n} = \lim_{n\to\infty} \frac{3n^2}{\ln(2)2^n} = \lim_{n\to\infty} \frac{6n}{\ln^2(2)2^n} = 0$$

$$\lim_{n\to\infty} \frac{2^n}{4^n} = \lim_{n\to\infty} \left(\frac{1}{2}\right)^n = 0$$

Hence:

$$O(10) \subset O(\ln(n)) \subset O(n) \ldots \subset O(4^n)$$

3)

- Set first and last elements to ~~first~~ low and high indexes

- Calculate midpoint

- Create left and right bounds sublists

- Loop through sublists until $L[i] = i$ or -1 for if false

procedure: findFixedPoint $(L[0:n-1], n)$
input: $L[0:n-1]$ (sorted list)
output: fPI (index of fixed point)

```
lowI ← 0
highI ← n-1
while lowI ≤ highI
        fPI ← (lowI + highI)/2
        if L[fPI] < fPI
                lowI ← fPI + 1
        else if L[fPI] > fPI
                highI ← fPI - 1
        else return fPI
end while
return -1
end ~~findPoint~~ findFixedPoint
```

Best Case: $B(n)$ is when first fixed point determined is where $L[i] = i$

Worst case: $W(n)$ is $O(\log(n))$ as it would return -1 or the last determined fixed point.

4a)

$$\begin{pmatrix} fib(n) \\ fib(n+1) \end{pmatrix} = \begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix}^N \begin{pmatrix} 0 \\ 1 \end{pmatrix}$$

$$\begin{pmatrix} fib(0) \\ fib(0+1) = fib(1) \end{pmatrix} = \begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix}^0 \begin{pmatrix} 0 \\ 1 \end{pmatrix}$$

$$\begin{pmatrix} fib(0) \\ fib(1) \end{pmatrix} = \begin{pmatrix} 0 \\ 1 \end{pmatrix} \quad \text{as } A^0 = I$$

$$\begin{pmatrix} fib(k) \\ fib(k+1) \end{pmatrix} = \begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix}^k \begin{pmatrix} 0 \\ 1 \end{pmatrix}$$

$$\begin{pmatrix} fib(k+1) \\ \hline fib(k+2) \end{pmatrix} = \begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix}^k \begin{pmatrix} 0 \\ 1 \end{pmatrix}$$

4b)

- Check base cases, $n=0$ returns 0
  $n=1$ returns 1
- Compute $fib(n/2)$ and ~~fib(n+1/2)~~
  $fib(n+1/2)$
- Raise to n^{th} power $\begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix}$
- Handle $\begin{pmatrix} 0 \\ 1 \end{pmatrix}$ multiplications

5a) $W(n) = 3W(n/3) + 2N/3$

$W(n) = 3W(n/3) + N/3$

5b) $3W(n/3) + 2n/3$

$= 3[3W(n/9) + 2/3(n/3)] + 2n/3$

$= 3^2 W(n/9) + 2(n/3) + 2N/3$

$W(n) = 3^k W(n/3^k) + k\left(\frac{2n}{3}\right)$

$n/3^k = 1 \rightarrow k = \log_3 n$

$W(n) = 3\log_3 n \, W(1) + \log_3 n \cdot 2n/3$

$W(n) = n(W(1)) + 2n\log_3 W$

5c) Mergesort is simplier to implement despite both trisort and mergesort having complexities of $O(n\log n)$, thus mergesort is preferred.